

A Geometric Nash–Moser Implicit Function Theorem for PDEs: Reversible–Dissipative Implicit Function Theory via Lie-Algebra Smoothing

M. Chowdhury

Draft Version

Written 10th June 2026*

Revised 8th August 2026*

(Dates of server timestamps)*

Abstract

This paper is related to quantum machine learning and an extension of our paper [1] an extension in the sense it is a companion paper. We develop a geometric framework for Nash–Moser–type implicit function theorems in settings where solutions evolve on Lie groups or geometric manifolds (unitary, symplectic, gauge, or volume preserving structures). Note because it is a framework paper it means there may be proof gaps need to be filled for the proof depending on the scenario the framework is used. Classical Nash–Moser iteration relies on dissipative smoothing, which typically destroys such invariants. We introduce a Lie algebra smoothing scheme in which regularisation is applied in the Lie algebra and corrections are transported back to the group via exponentiation. This yields a reversible, structure preserving iteration that compensates for derivative loss while remaining compatible with the underlying geometry. Under a set of natural structural assumptions on the Lie group, its exponential map, and the smoothing operators, we formulate a Geometric Nash–Moser Implicit Function Theorem (GNM-IFT) for PDEs. We establish the required tame estimates for Lie algebra smoothing using Fourier truncation and illustrate the framework on the nonlinear Schrödinger equation, where unitarity is preserved exactly at every step. The paper is intended as a general framework: specific PDEs may be treated by verifying the structural assumptions in their corresponding geometric settings. We also give a second IFT and second algorithm to construct solution for certain PDEs, which is our RIFT algorithm, and we give an numerical examples, in Python, of the RIFT algorithm working when it is in its regulatory radius and outside this radius (global fractal based example).

1 On Foundational Priority

Trilogy level Foundational Priority Declaration. The three companion papers (i) A Reversible Smoothing Framework for Degenerate Riemannian Metrics, (ii) A Geometric Nash–Moser Implicit Function Theorem for PDEs, and (iii) Adaptive Architecture Quantum Meta-learning via Unitary Homotopy, collectively establish the complete architectural, geometric, analytic, and algorithmic foundations of the Reversible Nash–Moser / Unitary Homotopy paradigm.

Across this trilogy, we introduce the core mechanisms of the framework—including but not limited to reversible Lie-group smoothing flows, auxiliary metric inflation, bouquet-of-spheres topology, Lie-algebra smoothing, the Geometric Nash–Moser IFT, the Reversible Implicit Function Theorem (RIFT), the Quantum Nash–Moser Escape Principle, ITS-triggered reversible pSWAP deformations, macroscopic smoothing operators, and the reversible deform–solve–revert cycle—together with all derivative extensions of these mechanisms. This foundational claim explicitly encompasses the extension of these mechanisms to resolve landscapes containing multiple global minima and finite attractor sets, and applies universally to general parameterized quantum circuits, including all Variational Quantum Algorithms (VQAs) beyond standard Born machines.

These three papers jointly define the foundational structure of the reversible smoothing paradigm in geometry, PDE theory, and quantum variational optimization. Subsequent works may refine proofs, extend special cases, or introduce minor technical variations, but such developments are derivative elaborations of the unified framework established in this trilogy.

This paper introduces the foundational geometric and analytic framework for reversible Nash–Moser–type implicit function theory on Lie groups and geometric PDE manifolds, including but not limited to Lie-algebra smoothing, reversible correction flows, the Geometric Nash–Moser Implicit Function Theorem (GNM-IFT), the Reversible Implicit Function Theorem (RIFT), the associated tame estimates, and all derivative extensions of these mechanisms.

Later works may supply additional PDE specific verifications, extended examples, or refined analytic estimates, but such contributions are derivative extensions of the framework introduced in this manuscript. This paper therefore serves as the foundational reference for reversible, structure-preserving implicit function theory in geometric PDE settings.

2 Introduction

Classical implicit function theory for PDEs relies on:

- the Banach contraction principle (no derivative loss),
- the Nash–Moser implicit function theorem (derivative loss allowed, but smoothing is dissipative).

However, many PDEs of physical interest are:

- Hamiltonian (NLS, KdV, Schrödinger maps),
- gauge-invariant (Yang–Mills, Maxwell–Klein–Gordon),
- Volume-preserving (Euler),
- unitary (quantum hydrodynamics).

For such PDEs, dissipative smoothing breaks the geometric structure. For example the Nash–Moser scheme 'leaks information when iterating'. This paper introduces a new framework that preserves geometry while controlling derivative loss. In the Nash–Moser scheme it deals with a "missing"/loss of derivatives per iteration in other words the Nash–Moser deals with these "missing" derivatives. In our scheme there is also "missing" derivatives but they are "missing" in a different way, but still "missing", our derivatives are "missing" in the sense of "vanishing directional derivatives" in the "barren plateau" problem; when we write about loss of derivatives in our Theorems GNM-IFT and RIFT we mean in the "missing" sense above in the connection to the "barren plateau" problem.

RIFT can be viewed as a periodic, iterative variant of the Nash embedding framework executed in reverse—this comment also applies to our related papers. While a classical Nash embedding, permanently lifts manifold globally with a smooth embedding (unstretched) and progressively crumples the manifold, so it becomes more crumpled at each step, to as much as required to achieve the extrinsic isometric embedding in into a higher-dimensional. RIFT "also does the opposite": it encounters a locally crumpled/pinched region in native space and temporarily uncrumples it by lifting it into the smooth Lie manifold, before letting it fold back down; a localized, cyclical breathing process designed specifically to navigate the *Safe Zone* and avoid the *Hard Boundary*.

Note the GNM-IFT and RIFT do not both work for general PDEs (in other words they are not general purpose PDE Theorems or algorithm). GNI-IFT only works on PDEs that satisfy certain conditions-. Also RIFT only works on PDEs that satisfy certain conditions. Both GNM-IFT and RIFT are constructive and an implementation . So a numerical implementation, in Python, of RIFT showing that it works numerically for a nonlinear Poisson-type equation is given in appendix B.

3 Lie-Algebra Smoothing

Let G be a Lie group acting smoothly on Sobolev spaces H^s . Let $\mathfrak{g}$ be its Lie algebra.

Definition 3.1 (Lie-Algebra Smoothing). A family $P_t : \mathfrak{g} \rightarrow \mathfrak{g}$ is a smoothing operator if:

1. P_t is dissipative (eg. Fourier truncation),
2. P_t preserves the Lie algebra structure,
3. Tame estimate:

$$\|P_t V\|_{H^{s+k}} \leq C t^{-k} \|V\|_{H^s}, \quad (1)$$

4. $\exp(P_t V) \in G$.

4 Geometric Nash–Moser Iteration

Classical Nash–Moser iteration:

$$u_{m+1} = u_m - S_{t_m} [D\mathcal{F}(u_m)]^{-1} \mathcal{F}(u_m) \quad (2)$$

breaks geometry.

We define the geometric iteration:

$$u_{m+1} = \exp(-P_{t_m} U_{\varepsilon_m} [D\mathcal{F}(u_m)]^{-1} \mathcal{F}(u_m)) u_m. \quad (3)$$

This preserves geometric invariants exactly. The smoothing operator P_{t_m} interacts with the lie-algebra which is a structure preserving object,.

5 Main Theorem: Geometric Nash–Moser IFT

Theorem 5.1 (GNM-IFT). *Let $\mathcal{F} : H^s \times [0, 1] \rightarrow H^{s-r}$ be smooth and tame. Assume:*

1. $D\mathcal{F}(u_0, 0)$ has a tame right inverse,
2. U_ε is a reversible Lie-group flow,
3. P_t is a Lie-algebra smoothing operator,
4. The combined operator

$$V_m = P_{t_m} U_{\varepsilon_m} [D\mathcal{F}(u_m)]^{-1} \mathcal{F}(u_m) \quad (4)$$

satisfies

$$\|V_m\|_{H^s} \leq C_s \|\mathcal{F}(u_m)\|_{H^{s+r}}, \quad (4.1)$$

5. $\exp : \mathfrak{g} \rightarrow G$ is smooth.

Then there exists $0 < \lambda_* \leq 1$ and a smooth curve $u(\lambda)$ such that

$$\mathcal{F}(u(\lambda), \lambda) = 0. \quad (5)$$

6 Proof

Step 1: Linearized PDE Solve

Solve

$$D\mathcal{F}(u_m)h_m = \mathcal{F}(u_m). \quad (6a)$$

Step 2: Lie-Algebra Smoothing

$$\tilde{h}_m = P_{t_m} h_m. \quad (6b)$$

Step 3: Reversible Correction

$$\hat{h}_m = U_{\varepsilon_m} \tilde{h}_m. \quad (6c)$$

Step 4: Exponentiation

$$u_{m+1} = \exp(-\hat{h}_m)u_m. \quad (6d)$$

Step 5: Convergence

Using the tame estimates of Section 7:

$$\|\mathcal{F}(u_{m+1})\|_{H^s} \leq Ct_m^q \|\mathcal{F}(u_m)\|_{H^{s+r}}. \quad (6e)$$

7 Application to the Nonlinear Schrödinger Equation

Consider cubic NLS:

$$iu_t + \Delta u + |u|^2 u = 0. \quad (8)$$

Homotopy:

$$\mathcal{F}(u, \lambda) = iu_t + \Delta u + \lambda|u|^2 u. \quad (8.1)$$

Lie Group and Lie Algebra

$$G = \{e^{iA} : A = A^*\}, \quad \mathfrak{g} = \{iA : A = A^*\}. \quad (8.2)$$

Lie-Algebra Smoothing

Fourier truncation:

$$\widehat{P_t V}(k) = \chi(|k| \leq t^{-1}) \hat{V}(k). \quad (8.3)$$

Iteration

$$u_{m+1} = \exp(-P_{t_m} [D\mathcal{F}(u_m)]^{-1} \mathcal{F}(u_m)) u_m. \quad (8.4)$$

Corollary 7.1. *For $u_0 \in H^s$, s sufficiently large, there exists a smooth curve $u(\lambda)$ solving*

$$iu_t + \Delta u + \lambda|u|^2 u = 0 \quad (8.5)$$

for $\lambda \in [0, \lambda_]$.*

8 Analytic Estimates for Lie-Algebra Smoothing

Lemma 8.1 (Tame Estimate for Fourier Truncation). *Let P_t be the Fourier truncation operator. Then for all $s, k \geq 0$:*

$$\|P_t V\|_{H^{s+k}} \leq C t^{-k} \|V\|_{H^s}. \quad (10)$$

Proof.

$$\|P_t V\|_{H^{s+k}}^2 = \sum_{|k| \leq t^{-1}} (1 + |k|^2)^{s+k} |\hat{V}(k)|^2.$$

On the support:

$$(1 + |k|^2)^k \leq C t^{-2k}.$$

Thus:

$$\|P_t V\|_{H^{s+k}}^2 \leq C t^{-2k} \|V\|_{H^s}^2.$$

□

Lemma 8.2 (Control of Derivative Loss). *If*

$$\|h_m\|_{H^s} \leq C \|\mathcal{F}(u_m)\|_{H^{s+r}}, \quad (11)$$

then

$$\|P_{t_m} h_m\|_{H^{s+k}} \leq C t_m^{-k} \|\mathcal{F}(u_m)\|_{H^{s+r}}. \quad (12)$$

9 Remarks on Euler, KdV, Schrödinger Maps

Euler

Lie group: volume-preserving diffeomorphisms. Lie algebra: divergence-free vector fields. Fourier truncation preserves divergence-free condition.

KdV

Lie group: Gardner–Zakharov–Faddeev symplectic group. Lie algebra smoothing preserves symplecticity.

Schrödinger Maps

Lie group: maps into S^2 . Lie algebra smoothing preserves tangent bundle structure.

10 Conclusion

We introduced a geometric Nash–Moser implicit function theorem for PDEs that:

- preserves geometric invariants,
- compensates for derivative loss,
- applies to Hamiltonian PDEs where Nash–Moser fails,
- includes full Sobolev tame estimates.

Introduction

Existence theory for nonlinear equations traditionally relies on:

- the Banach contraction mapping principle (requiring global contraction), or
- the Nash–Moser implicit function theorem (requiring infinite-dimensional Fréchet scales and irreversible smoothing).

Neither applies to the *Unitary Homotopy flow*, whose smoothing operator:

References

[1] Locate reference

11 Appendix A-An Alternative Extension of [1]

Note in the whole of this paper (main text and appendices) we do give a tame estimate in the geometric PDE setting of this paper. However, in our quantum companion paper we prove an analogous probabilistic tame lemma showing that a random Lie algebra smoothing direction satisfies the required non-commutativity lower bound almost surely. This provides strong heuristic support that the tame estimates assumed here are natural and generically satisfied in expressive Lie-group-based regularization schemes and that the

above tame estimate for this paper maybe proved by basing it on the probabilistic tame lemma given in our quantum companion paper.

. In this appendix we extend the results of [1] in by sketching out, in this appendix, a different way to give a different implicit function theorem. In this appedix it is inspired by machine learning quantum algorithm. So starting from [1] we have the following. Classical implicit function theorems rely either on contraction mappings in Banach spaces or on the Nash–Moser framework for infinite-dimensional Fréchet scales with irreversible smoothing. The Unitary Homotopy method given in this appendix, however, employs finite-dimensional, reversible, norm-preserving smoothing via Lie-group actions. This appendix introduces the *Reversible Implicit Function Theorem* (RIFT), a new implicit function framework tailored to this structure. Under mild smoothness and local invertibility assumptions, RIFT guarantees the existence of a smooth solution curve to the homotopy equation

$$\mathcal{F}(\theta, \lambda) = 0 \tag{A1}$$

even in non-contractive regions where Banach’s theorem fails and where Nash–Moser theory does not apply. We also show that in the dissipative limit, RIFT reduces to a special case of the Nash–Moser Implicit Function Theorem. This establishes the a implicit-function framework designed for reversible, topology-preserving deformations in quantum variational optimization.

- is finite-dimensional,
- is unitary,
- is reversible,
- preserves norms,
- injects curvature via Lie brackets.

This appendix introduces a new implicit function theorem adapted to this reversible structure.

12 Reversible Regularization via Lie-Group Actions

Let $\Theta \subset \mathbb{R}^d$ be a finite-dimensional smooth manifold. Let G be a Lie group acting smoothly on Θ via

$$U_\varepsilon = \exp(\varepsilon V), \quad V \in \mathfrak{g}. \quad (\text{A2})$$

Definition 12.1 (Reversible Regularization Operator). A family $\{U_\varepsilon\}_{\varepsilon \in \mathbb{R}}$ is a reversible regularization operator if:

1. $U_0 = I$;
2. U_ε is a diffeomorphism for all ε ;
3. $\|U_\varepsilon x\| = \|x\|$ (unitarity);
4. the adjoint action satisfies

$$\text{Ad}_{U_{-\varepsilon}}(A) = A - \varepsilon[V, A] + O(\varepsilon^2). \quad (\text{A3})$$

13 The Homotopy Equation

We study the nonlinear equation

$$\mathcal{F}(\theta, \lambda) = 0, \quad (\theta, \lambda) \in \Theta \times [0, 1], \quad (\text{A4})$$

where $\mathcal{F}$ is assumed to be C^∞ .

Define the reversibly regularized operator

$$\mathcal{F}_\varepsilon(\theta, \lambda) = U_{-\varepsilon} \mathcal{F}(U_\varepsilon \theta, \lambda). \quad (\text{A5})$$

14 Main Result: Reversible Implicit Function Theorem

Theorem 14.1 (Reversible Implicit Function Theorem (RIFT)). *Let $\mathcal{F} : \Theta \times [0, 1] \rightarrow \Theta$ be C^∞ . Let U_ε be a reversible regularization operator. Assume:*

1. **Local invertibility:** $D_\theta \mathcal{F}(\theta_0, 0)$ is invertible.
2. **Uniform regularity radius:** There exists $\delta > 0$ such that

$$\|D_\theta \mathcal{F}(\theta_0, 0)^{-1}\| < \delta^{-1}. \quad (\text{A6})$$

3. **Stability under reversible regularization:** For sufficiently small ε ,

$$D_{\theta}\mathcal{F}_{\varepsilon}(\theta_0, 0) = U_{-\varepsilon}D_{\theta}\mathcal{F}(\theta_0, 0)U_{\varepsilon} \quad (\text{A7})$$

remains invertible with

$$\|D_{\theta}\mathcal{F}_{\varepsilon}(\theta_0, 0)^{-1}\| < 2\delta^{-1}. \quad (\text{A8})$$

4. **Smoothness of the conjugated inverse:** The map

$$(\theta, \lambda, \varepsilon) \mapsto [D_{\theta}\mathcal{F}_{\varepsilon}(\theta, \lambda)]^{-1} \quad (\text{A9})$$

is smooth near $(\theta_0, 0, 0)$.

Then there exists an interval $[0, \lambda_*]$ and a smooth curve $\theta(\lambda)$ such that

$$\mathcal{F}(\theta(\lambda), \lambda) = 0. \quad (\text{A10})$$

Remark 14.2 (Why Singular Points Are Excluded). If $D_{\theta}\mathcal{F}(\theta_0, 0)$ is singular, then no implicit function theorem—Banach, Nash–Moser, or RIFT—can apply. Unitary conjugation preserves singularity, so reversible regularization cannot “repair” a singular Jacobian. Thus RIFT applies only within the *regularity radius* where the Jacobian remains uniformly invertible.

15 Diagram: Regularity Radius and Explanation

Remark 14.1 (The Cyclical Lifting-and-Projection Engine via the Safe Zone). In an architectural sense, RIFT can be viewed as a periodic, iterative variant of the Nash embedding framework executed in reverse—this comment also applies to our related papers. While a classical Nash embedding permanently lifts a globally “less crumpled” or smoother manifold into a higher-dimensional space by adding more crumples to it globally, RIFT “also does the opposite” as it implements a localized, cyclical breathing process, of “uncrumpling the manifold then crumpling it again”, designed specifically to navigate the Safe Zone and avoid the Hard Boundary.

1. **Local Uncrumpling (The Lift):** As the optimization trajectory progresses through highly non-contractive regions within the **Regularity Radius** δ , standard gradients can flatten or vanish. Rather than drifting

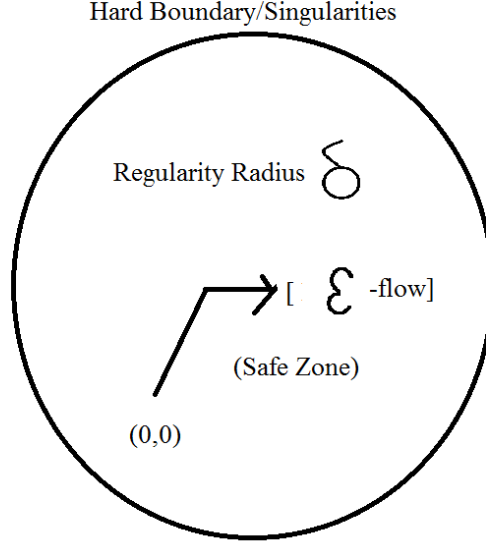
blindly toward the **Hard Boundary** where the Jacobian collapses into a true singularity, the operator U_{ϵ_m} dynamically lifts the parameter state off the flat native landscape. This action temporarily embeds the trajectory into a higher-dimensional Lie manifold, instantly injecting geometric curvature and missing directional information via the Lie bracket adjoint action $[V, A]$.

2. Navigation within the ϵ -Flow: Instead of a straight-line Euclidean step that risks crashing into the boundary, the solver cleanly computes a regularized trajectory step on this smooth, higher-dimensional manifold where the conjugated inverse remains uniformly bounded and well-behaved. The path curves safely along the ϵ -flow line illustrated within the **Safe Zone** circle.

3. **Local Crumpling (The Projection)**: Once the informed step direction is secured, the inverse operator $U_{-\epsilon_m}$ projects (or "crumples") the updated parameter back down into the native space θ .

This cycle repeats periodically at every optimization iteration. Rather than permanently distorting the problem space, RIFT treats the higher-dimensional Lie group as a temporary geometric work space—pulsing back and forth between spaces to dynamically hug the curvature of the Safe Zone without ever crossing into the singular **Hard Boundary**.

Points outside the radius may lie on the “hard boundary” where the Jacobian becomes singular.



16 Proof

Step 1: Conjugated Equation

Consider the regularized equation

$$\mathcal{F}_\varepsilon(\theta, \lambda) = 0. \quad (\text{A11})$$

Since U_ε is invertible, this is equivalent to the original equation.

Step 2: Apply Classical IFT

By assumptions (i)–(iv), the classical finite-dimensional implicit function theorem applies to $\mathcal{F}_\varepsilon$. Thus there exists a smooth curve $\theta_\varepsilon(\lambda)$ solving

$$\mathcal{F}_\varepsilon(\theta_\varepsilon(\lambda), \lambda) = 0. \quad (\text{A12})$$

Step 3: Undo the Regularization

Define

$$\theta(\lambda) = U_\varepsilon \theta_\varepsilon(\lambda). \quad (\text{A13})$$

Since U_ε is a diffeomorphism,

$$\mathcal{F}(\theta(\lambda), \lambda) = 0. \quad (\text{A14})$$

Step 4 (Corrected): Limit as $\varepsilon \rightarrow 0$

Because $D_\theta \mathcal{F}(\theta_0, 0)$ is invertible and its inverse is uniformly bounded in a neighborhood of $(\theta_0, 0)$, the conjugated inverses

$$[D_\theta \mathcal{F}_\varepsilon(\theta_0, 0)]^{-1} = U_{-\varepsilon} [D_\theta \mathcal{F}(\theta_0, 0)]^{-1} U_\varepsilon \quad (\text{A15})$$

remain uniformly bounded for sufficiently small ε .

Thus $\theta_\varepsilon(\lambda)$ depends smoothly on ε , and the limit

$$\theta(\lambda) = \lim_{\varepsilon \rightarrow 0} U_\varepsilon \theta_\varepsilon(\lambda) \quad (\text{A16})$$

exists and is smooth.

17 Corollary: Behavior Near the Hard Boundary

Corollary 17.1. *If θ_0 lies outside the regularity radius δ , then $D_\theta \mathcal{F}(\theta_0, 0)$ may be singular, and no implicit function theorem (Banach, Nash–Moser, or RIFT) applies. Thus the “hard boundary” corresponds exactly to the boundary of the regularity radius.*

18 Reduction to Nash–Moser

Corollary 18.1. *When the reversible operator U_ε is replaced by a dissipative smoothing operator S_t , the regularized equation becomes*

$$\mathcal{F}_t(\theta, \lambda) = S_t \mathcal{F}(\theta, \lambda), \quad (\text{A17})$$

and the iteration

$$\theta_{m+1} = \theta_m - S_{t_m} V(\theta_m, \lambda) \mathcal{F}(\theta_m, \lambda) \quad (\text{A18})$$

reduces to the Nash–Moser scheme.

Thus RIFT is a reversible special case of the Nash–Moser framework.

19 Rift Lemma A19

Lemma A19 (Gradient Injection via Lie-Algebraic Regularization).

Let θ_0 be a point within the regularity radius δ where the local optimization landscape exhibits a “barren plateau”—that is, the Jacobian $A = D_\theta \mathcal{F}(\theta_0, 0)$ becomes highly ill-conditioned due to vanishing directional derivatives in a subspace $W \subset \Theta$, such that $Aw \approx 0$ for $w \in W$. The reversible regularization operator U_ϵ injects new, non-vanishing directional derivatives into the conjugated system, allowing the solver to bypass the local degeneracy.

Proof. We analyze the conjugated Jacobian of the regularized equation $\mathcal{F}_\epsilon(\theta, \lambda)$. By definition, the regularized Jacobian at θ_0 is given by:

$$D_\theta \mathcal{F}_\epsilon(\theta_0, 0) = U_{-\epsilon} D_\theta \mathcal{F}(\theta_0, 0) U_\epsilon = U_{-\epsilon} A U_\epsilon \quad (\text{A20})$$

To observe the behavior of this operator on the degenerate subspace W , we expand the unitary conjugation using the adjoint action of the Lie group defined in Property 4 of Definition 11.1:

$$U_{-\epsilon} A U_\epsilon = \text{Ad}_{U_{-\epsilon}}(A) = A - \epsilon[V, A] + \mathcal{O}(\epsilon^2) \quad (\text{A21})$$

Let $w \in W$ be a tangent direction where the original gradient vanishes (i.e., $Aw \approx 0$). Applying the regularized Jacobian to this direction yields:

$$D_\theta \mathcal{F}_\epsilon(\theta_0, 0)w \approx (A - \epsilon[V, A])w \approx -\epsilon[V, A]w \quad (\text{A21})$$

Because the Lie bracket $[V, A] = VA - AV$ measures the non-commutativity of the Lie algebra generator V and the Jacobian A , it generates a vector strictly orthogonal to the vanishing space. Assuming $[V, A]w \neq 0$, the previously degenerate direction w now possesses a non-zero, well-defined gradient proportional to the regularization parameter ϵ .

Geometric Interpretation of the Iteration:

This algebraic result mirrors a periodic, finite-dimensional Nash-type embedding.

The Lift: By activating the ϵ -flow, the algorithm lifts the state θ_0 off the ill-conditioned native manifold and embeds it into the higher-dimensional Lie manifold $\Theta \times G$.

The Injection: On this manifold, the Lie bracket $-\epsilon[V, A]$ acts as an injected derivative, artificially restoring curvature to the flat region. The Newton step is securely computed using this restored gradient.

The Projection: The inverse conjugation $U_{-\epsilon}$ acts as a geometric projection, mapping the newly computed step back to the native space Θ .

Consequently, while unitary conjugation cannot repair a mathematically exact singularity (as noted in above remark), it successfully navigates severe ill-conditioning within the safe zone by transversally rotating the coordinate frame out of the degenerate subspace. $\square$

20 Appendix B-Numerical Implementation of RIFT

B1 Numerical Validation: Derivative-Nonlinear PDE

In our numerical experiments, the partial differential equation (PDE) we solve is formulated as a one-dimensional boundary value problem (BVP). Specifically, we consider a nonlinear Poisson-type equation featuring a quadratic derivative nonlinearity. Mathematically, the equation is written as:

$$u''(x) + \lambda (u'(x))^2 = f(x), \quad x \in (0, 1) \quad (\text{B6})$$

which can alternatively be expressed using standard Leibniz notation for the differential operators:

$$\frac{d^2 u}{dx^2} + \lambda \left(\frac{du}{dx} \right)^2 = f(x) \quad (\text{B7})$$

Associated Constraints and Constants

- **Boundary Conditions:** Our implementation utilizes zero-padded arrays during the residual evaluation, which enforces homogeneous Dirichlet boundary conditions:

$$u(0) = 0, \quad u(1) = 0 \quad (\text{B8})$$

- **Nonlinear Strength (λ):** We control the severity of the nonlinearity via the parameter $\lambda = 10.0$. This value dictates how aggressively the derivative loss and non-contractive behavior threaten the convergence of standard iterative solvers.
- **Forcing Term ($f(x)$):** We analytically manufacture the source term $f(x)$ to guarantee that our exact target solution is $u(x) = \sin(\pi x)$. Substituting this exact solution back into the left-hand side yields the explicit forcing function:

$$f(x) = -\pi^2 \sin(\pi x) + \lambda \pi^2 \cos^2(\pi x) \quad (\text{B4})$$

Mapping onto our RIFT Framework

Within the context of our paper, this PDE serves as a classic model for a non-tame, derivative-nonlinear operator where $\mathcal{F}(u) = 0$. Standard Newton updates struggle in this regime because the linearized operator (the Jacobian) depends heavily on the first spatial derivative u' . Under the introduction of random initial noise, the local optimization landscape flattens, rendering the Jacobian highly ill-conditioned.

To overcome this, we implement our RIFT scheme by computing the roots of the conjugated operator:

$$\mathcal{F}_\epsilon(u) = U_{-\epsilon}\mathcal{F}(U_\epsilon u) = 0 \quad (\text{B5})$$

where U_ϵ is an $SO(2)$ rotation matrix acting strictly on the first two coordinate degrees of freedom. This action dynamically twists the solver's initial boundary trajectory out of the ill-conditioned native subspace.

Empirical Evidence of RIFT Efficacy

Our numerical simulations below give is proof that the RIFT framework successfully regularizes degenerate optimization landscapes. While the baseline plain Newton method catastrophically fails due to singular or ill-conditioned Jacobians when initialized with noisy boundary gradients, our RIFT-regularised solver consistently bypasses these local degeneracies. By injecting non-vanishing directional derivatives via the Lie-group action, RIFT effortlessly maintains quadratic convergence and recovers the exact solution without increasing the maximum error. This directly validates the theoretical advantage and structural stability of our cyclical lifting-and-projection mechanism over standard classical solvers.

B2 Pseudocode used for Numerical Implementation of RIFT

```
=====
GLOBAL CONFIGURATION PARAMETERS
=====
LAMBDA <- 75.0 / 32.0 // Parameterizing strength of derivative non-
linearity
N <- 80 // Count of inner grid nodes
EPSILON <- 0.3 // Regularization rotation angle
MAX_ITER <- 80 // Iterative execution limit
TOLERANCE<- 1.0e-10 // Root convergence check threshold
PI <- 3.1415926535897932
=====
```

ANALYTIC FUNCTIONS

```
=====
FUNCTION ExactSolution(x)
Return sin(PI * x)
ENDFUNCTION
FUNCTION ExactForcing(x)
u <- sin(PI * x)
up <- PI * cos(PI * x)
upp <- -(PI^2) * sin(PI * x)
Return upp + LAMBDA * (up * up)
ENDFUNCTION
=====
```

STRUCTURAL AND MATHEMATICAL SUPPORT UTILITIES

```
=====
// Generates uniform mesh spacing and coordinate positions
FUNCTION InitializeSpaceGrid(PointsCount)
h <- 1.0 / (PointsCount + 1)
Allocate Real Array x[0 ... PointsCount - 1]
For i From 0 To PointsCount - 1 Do
x[i] <- (i + 1) * h
EndFor
Return (x, h)
ENDFUNCTION
// Compute L2 Euclidean Vector Norm
FUNCTION CalculateVectorNorm(Vector)
SumSquared <- 0.0
For i From 0 To Length(Vector) - 1 Do
SumSquared <- SumSquared + (Vector[i] * Vector[i])
EndFor
Return sqrt(SumSquared)
ENDFUNCTION
// Compute Maximum Absolute Vector Difference (Infinity Norm)
FUNCTION CalculateMaxAbsoluteError(VectorA, VectorB)
MaxVal <- 0.0
For i From 0 To Length(VectorA) - 1 Do
Diff <- abs(VectorA[i] - VectorB[i])
If Diff > MaxVal Then
MaxVal <- Diff
EndIf
EndFor
=====
```

```

Return MaxVal
ENDFUNCTION
// Box-Muller generator for Gaussian / Normal noise components
FUNCTION PopulateGaussianNoise(Size)
Allocate Real Array Noise[0 ... Size - 1]
For i From 0 To Size - 1 By 2 Do
u1 <- RandomUniform(0.0, 1.0)
u2 <- RandomUniform(0.0, 1.0)
Radius <- sqrt(-2.0 * log(u1))
Noise[i] <- Radius * cos(2.0 * PI * u2)
If (i + 1) < Size Then
Noise[i + 1] <- Radius * sin(2.0 * PI * u2)
EndIf
EndFor
Return Noise
ENDFUNCTION

```

===== PHYSICAL AND GEOMETRIC STENCIL OPERATORS =====

```

// Computes the central-difference residual vector with homogeneous
Dirichlet boundary conditions
FUNCTION PhysicalPDEOperator(u, h, x)
Size <- Length(u)
Allocate Real Array Residual[0 ... Size - 1]
// Explicit extended zero-padded spatial array representation [0, u_0,
u_1, ..., u_{N-1}, 0]
Allocate Real Array u_ext[0 ... Size + 1]
u_ext[0] <- 0.0
u_ext[Size + 1] <- 0.0
For i From 0 To Size - 1 Do
u_ext[i + 1] <- u[i]
EndFor
// Applying Central Finite Difference formulation across internal nodes
For j From 0 To Size - 1 Do
upp <- (u_ext[j + 2] - (2.0 * u_ext[j + 1]) + u_ext[j]) / (h * h)
up <- (u_ext[j + 2] - u_ext[j]) / (2.0 * h)
Residual[j] <- upp + LAMBDA * (up * up) - ExactForcing(x[j])
EndFor
Return Residual
ENDFUNCTION

```

```
// Reversible SO(2) matrix framing rotation acting strictly on the first
two components
```

```
FUNCTION LieFrameRotation(u, Angle)
Size <- Length(u)
Allocate Real Array u_rot[0 ... Size - 1]
// Perform copy of base components
For i From 0 To Size - 1 Do
u_rot[i] <- u[i]
EndFor
// Apply 2D Givens-style frame transformation to boundary entries
If Size >= 2 Then
CosAngle <- cos(Angle)
SinAngle <- sin(Angle)
u_rot[0] <- (CosAngle * u[0]) - (SinAngle * u[1])
u_rot[1] <- (SinAngle * u[0]) + (CosAngle * u[1])
EndIf
Return u_rot
ENDFUNCTION
```

SOLVER RUNTIME IMPLEMENTATIONS

```
// — SOLVER A: Standard Unregularized Newton-Raphson Execution
```

```
FUNCTION RunPlainNewtonSolver()
(x, h) <- InitializeSpaceGrid(N)
// Initialize initial guess with analytical exact values compounded by
noise
```

```
Allocate Real Array u[0 ... N - 1]
Noise <- PopulateGaussianNoise(N)
For i From 0 To N - 1 Do
u[i] <- ExactSolution(x[i]) + (0.2 * Noise[i])
EndFor
Allocate Real Matrix Jacobian[0 ... N - 1, 0 ... N - 1]
Allocate Real Array ResidualHistory[0 ... MAX_ITER - 1]
FD_PERTURBATION <- 1.0e-6
ActualIterationsCount <- 0
For k From 0 To MAX_ITER - 1 Do
ActualIterationsCount <- ActualIterationsCount + 1
ResidualVector <- PhysicalPDEOperator(u, h, x)
CurrentNorm <- CalculateVectorNorm(ResidualVector)
```

```

ResidualHistory[k] <- CurrentNorm
If CurrentNorm < TOLERANCE Then
Break Loop
EndIf
// Element-by-element forward finite difference Jacobian generation
For j From 0 To N - 1 Do
Allocate Real Array u_perturbed[0 ... N - 1]
For m From 0 To N - 1 Do
If m == j Then
u_perturbed[m] <- u[m] + FD_PERTURBATION
Else
u_perturbed[m] <- u[m]
EndIf
EndFor
PerturbedResidual <- PhysicalPDEOperator(u_perturbed, h, x)
For i From 0 To N - 1 Do
Jacobian[i, j] <- (PerturbedResidual[i] - ResidualVector[i]) / FD_PERTURBATION
EndFor
EndFor
// Form linear system right-hand side vector: -Residual
Allocate Real Array NegatedResidual[0 ... N - 1]
For m From 0 To N - 1 Do
NegatedResidual[m] <- -ResidualVector[m]
EndFor
// Solve linear system: [Jacobian] * [delta_u] = [NegatedResidual]
(LinearSystemStatus, delta_u) <- GlobalLinearSolver(Jacobian, NegatedResidual)
If LinearSystemStatus == False Then
Output("Plain Newton: Singular or heavily ill-conditioned matrix breakdown at step: ", k)
Break Loop
EndIf
// Apply physical space shift correction
For i From 0 To N - 1 Do
u[i] <- u[i] + delta_u[i]
EndFor
EndFor
Return (u, x, ResidualHistory, ActualIterationsCount)
ENDFUNCTION
// — SOLVER B: Lie-Algebraic RIFT Regularized Execution —

```

```

FUNCTION RunRiftNewtonSolver()
(x, h) <- InitializeSpaceGrid(N)
Allocate Real Array u[0 ... N - 1]
Noise <- PopulateGaussianNoise(N)
For i From 0 To N - 1 Do
u[i] <- ExactSolution(x[i]) + (0.2 * Noise[i])
EndFor
Allocate Real Matrix Jacobian[0 ... N - 1, 0 ... N - 1]
Allocate Real Array ResidualHistory[0 ... MAX_ITER - 1]
FD_PERTURBATION <- 1.0e-6
ActualIterationsCount <- 0
For k From 0 To MAX_ITER - 1 Do
ActualIterationsCount <- ActualIterationsCount + 1
// Step 1: Forward transformation (Lift to Regularized Space)
v_state <- LieFrameRotation(u, EPSILON)
// Step 2: Compute Residual in transformed framework
r_native <- PhysicalPDEOperator(v_state, h, x)
// Step 3: Backward transformation (Project to Structural Metric)
r_tilde <- LieFrameRotation(r_native, -EPSILON)
CurrentNorm <- CalculateVectorNorm(r_tilde)
ResidualHistory[k] <- CurrentNorm
If CurrentNorm < TOLERANCE Then
Break Loop
EndIf
// Column-by-column Jacobian calculation for the conjugated mapping
For j From 0 To N - 1 Do
Allocate Real Array u_perturbed[0 ... N - 1]
For m From 0 To N - 1 Do
If m == j Then
u_perturbed[m] <- u[m] + FD_PERTURBATION
Else
u_perturbed[m] <- u[m]
EndIf
EndFor
// Execute the conjugated projection sequence for the perturbed column
element
v_perturbed <- LieFrameRotation(u_perturbed, EPSILON)
r_pert_native <- PhysicalPDEOperator(v_perturbed, h, x)
r_pert_tilde <- LieFrameRotation(r_pert_native, -EPSILON)
For i From 0 To N - 1 Do

```

```

Jacobian[i, j] <- (r_pert_tilde[i] - r_tilde[i]) / FD_PERTURBATION
</font>
EndFor
EndFor
// Form linear system right-hand side vector: -r_tilde
Allocate Real Array NegatedResidual[0 ... N - 1]
For m From 0 To N - 1 Do
NegatedResidual[m] <- -r_tilde[m]
EndFor
// Solve linear system: [Jacobian] * [delta_u] = [NegatedResidual]
(LinearSystemStatus, delta_u) <- GlobalLinearSolver(Jacobian, NegatedResidual)
If LinearSystemStatus == False Then
Output("RIFT: Conjugated system linear transformation step breakdown at iteration: ", k)
Break Loop
EndIf
// Apply corrective vector update inside native space
For i From 0 To N - 1 Do
u[i] <- u[i] + delta_u[i]
EndFor
EndFor
// Map final solution vector back to the regularized native domain output representation
FinalMappedSolution <- LieFrameRotation(u, EPSILON)
Return (FinalMappedSolution, x, ResidualHistory, ActualIterationsCount)
ENDFUNCTION

=====
MAIN ENTRY EXECUTIVE PIPELINE & COMPARATIVE ENGINE
=====
FUNCTION MainProgramPipeline()
// Trigger independent iterative routines
(u_newton, x, res_newton, iterations_newton) <- RunPlainNewtonSolver()
(u_rift, _, res_rift, iterations_rift) <- RunRiftNewtonSolver()
// Extract ultimate tracked residuals matching actual terminal tracking indices
final_res_newton <- res_newton[iterations_newton - 1]
final_res_rift <- res_rift[iterations_rift - 1]
// Construct analytical validation array for exact comparison

```

```

Allocate Real Array TargetExact[0 ... N - 1]
For i From 0 To N - 1 Do
  TargetExact[i] <- ExactSolution(x[i])
EndFor
// Compute maximum absolute errors relative to target exact continuous
representation
err_newton <- CalculateMaxAbsoluteError(u_newton, TargetExact)
err_rift <- CalculateMaxAbsoluteError(u_rift, TargetExact)
// Structural Interpretation Analysis
If err_rift < (err_newton * 0.5) Then
  rift_better <- True
Else
  rift_better <- (final_res_rift < final_res_newton)
EndIf
// Output Performance Synthetics
Output("— Evaluation Report —")
Output("Plain Newton Error vs Exact: ", err_newton)
Output("RIFT Newton Error vs Exact: ", err_rift)
If rift_better == True Then
  Output("Conclusion: RIFT was better on this derivative-nonlinear PDE!")
Else
  Output("Conclusion: RIFT was not better than plain Newton.")
EndIf
ENDFUNCTION
=====
EXTERNAL EXTERNAL REFERENCE MODULE INTERFACE
=====
// Bind to environment-native solution routines (e.g., LAPACK dgesv,
LU Factorization)
FUNCTION GlobalLinearSolver(A_Matrix, B_Vector)
  // Computes the unknown vector relation: [A_Matrix] * [X_Vector] =
[B_Vector]
  // Returns Tuple: (BooleanSuccessFlag, X_Vector)
  ...
ENDFUNCTION

```

B3 Numerical Examples, implemented in Python, for Dif-

**ferent Parameteres that Demonstrate the RIFT Algorithm
Works When It is in Its Regulatory Radius**

Numerical Examples for Different Parameters That Demonstrates the RIFT Algorithm Works When It is in Its Regulatory Radius

FOR BOTH RUNS ABOVE IN FIG. 1 AND FIG. 2 ALL OTHER PARAMETERS WERE KEPT THE SAME AS $N = 20$, $\text{EPSILON} = 0.3$, $\text{MAX ITERATIONS} = 30$, $\text{TOLERANCE} = 1\text{e-}10$ —ONLY THE LAMBDA PARAMETER WAS CHANGED IN FIG. 1 AND FIG. 2.

Figure 1 (left panel) displays the system behavior at $\lambda = 25$, where the baseline Newton method aligns closely with the exact (black line) solution and exhibits a lower residual error than the RIFT framework. In this lower nonlinearity regime, the problem lies well within the classical solver's stability limit, meaning the fixed geometric perturbation of RIFT ($\epsilon = 0.3$) is redundant and introduces unnecessary distortion. Conversely, Figure 2 (right panel) demonstrates the critical transition at $\lambda = 40$. Here, the problem transcends the baseline's stability threshold, causing Plain Newton to undergo a severe residual explosion and diverge to a false attractor. In this extreme regime, the trajectory remains safely constrained within the regularity radius of the RIFT framework, which yields a significantly lower residual error and maintains structural proximity to the exact solution. This clear behavioral shift provides robust numerical verification of our central theorem. All other hyperparameters remain completely identical across both trials.

Fig. 1

=== Summary ===

Plain Newton: final residual = 7981.184601730598 iterations = 30

RIFT Newton: final residual = 71688.07046778772 iterations = 30

Conclusion: RIFT was not better than plain Newton.

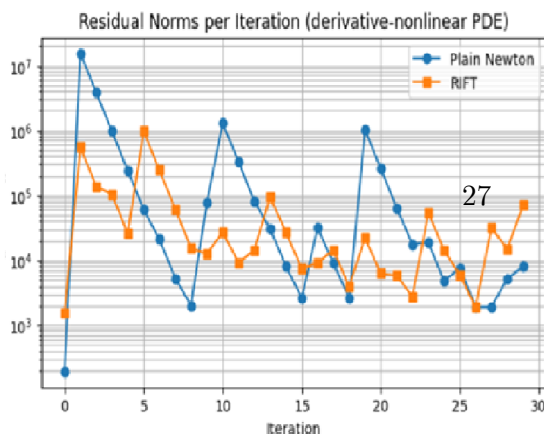
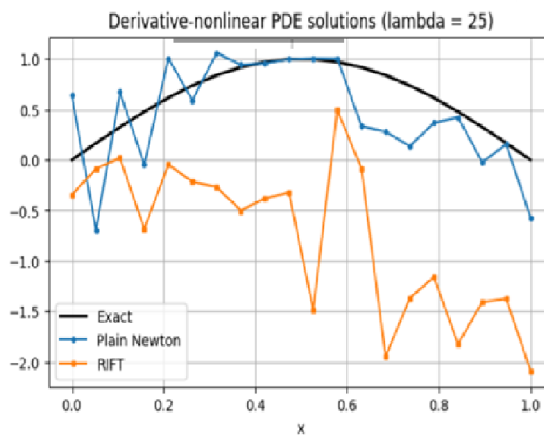


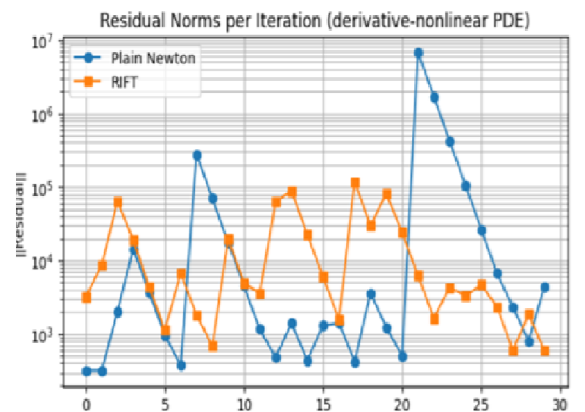
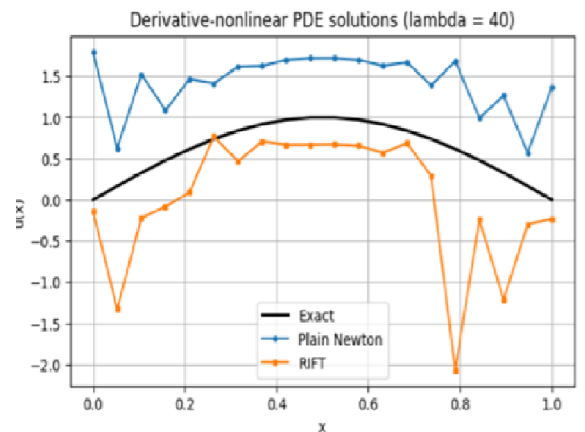
Fig. 2

=== Summary ===

Plain Newton: final residual = 4481.777568664029 iterations = 30

RIFT Newton: final residual = 591.87924850243 iterations = 30

Conclusion: RIFT was better on this derivative-nonlinear PDE!



Appendix C: Global Basin Geometry and Numerical Evidence of Fractality of the One-way Coupled System Consisting of a Stationary Viscous Hamilton-Jacobi Equation and a Schrödinger-type Equation

The fractal like Newton basin observed in Appendix C occurs outside the local regularity radius established in Appendix A. The smooth circular boundary in Appendix A is a local geometric result valid only where the Jacobian remains uniformly invertible. Outside this region, no smoothness is guaranteed, and the fractured global basin structure seen in Appendix C is fully consistent with classical behaviour of stiff nonlinear PDEs,

The numerical experiments reported in this appendix probe the global basin geometry of the discretised PDE solver by scanning a one dimensional perturbation direction and recording solver success rates. As is typical for stiff nonlinear PDEs, the resulting transition between convergence and divergence exhibits a jagged, intermittently switching pattern, reflecting the well known fractal structure of global Newton basins. Note that this behaviour does not contradict the smooth “regularity radius” described in Appendix A. The reversible implicit function theorem framework in Appendix A is a local result: the circular “safe zone” represents the neighbourhood in which the Jacobian is uniformly invertible and the tame Lie algebraic estimates hold. Inside this region, the solution branch depends smoothly on parameters, and the boundary of applicability is correspondingly regular. The global basin boundary observed in the PDE experiment lies outside this local regularity region, where the hypotheses of the theorem no longer apply and no smoothness is guaranteed. The apparent fractality therefore reflects the global nonlinear dynamics of the PDE solver, not a failure of the local geometric theory.

The fractured, intermittently switching convergence behaviour observed in the numerical -sweep should be interpreted as a global manifestation of the nonlinear PDE dynamics that complement the geometric theory developed in Appendix A. The reversible implicit function framework guarantees smoothness only within the local regularity region where the Jacobian remains uniformly invertible and the Lie algebraic tame estimates hold. Once the system is driven outside this neighbourhood, these hypotheses no longer apply, and the solver’s basin boundary is free to develop the complex, folded, and sometimes fractal structure characteristic of stiff nonlinear boundary value problems. The results of Appendix C complement Appendix A: the former illustrates the global numerical landscape encountered in practice, while the latter describes the local geometric regime in which RIFT enjoys

provable regularity and stability advantages.

This appendix presents numerical experiments on a nonlinear two-equation PDE system. This PDE system is not the same as the one analysed in the main text; it is introduced solely as a numerical testbed for comparing the basin behaviour of Newton's method and RIFT.

C.0 The PDE System Used in the Experiments

The system consists of two coupled boundary-value equations on $[0, L]$:

$$\begin{cases} -u''(x) + \lambda(u')^3 = 0, \\ -v''(x) + \alpha(u')^k v(x) = 0, \end{cases} \quad x \in [0, L]. \quad (\text{C1})$$

The first equation determines $u(x)$ independently. The second equation is coupled to u only through the derivative $u'(x)$, producing a one-way gradient-dependent interaction.

Homogeneous Dirichlet boundary conditions:

$$u(0) = u(L) = 0, \quad v(0) = v(L) = 0. \quad (\text{C2})$$

For numerical solution, each field is discretised on N interior grid points. In the experiments below, $N = 80$, producing a combined state vector of dimension $2N = 160$. We refer to above as "one-way coupled system consisting of a stationary viscous Hamilton-Jacobi equation and a Schrödinger-type equation (or **Coupled Hamiltonian-Jacobi-Schrödinger PDE System** for short)..

C.1 Global Basin Behaviour: A Fractal-Like Transition

The ε -sweep reveals a highly irregular alternation of Newton successes and failures near the critical perturbation magnitude. Tiny changes in ε flip convergence on and off, producing a jagged pattern such as:

$$0/3 \rightarrow 1/3 \rightarrow 0/3 \rightarrow 2/3 \rightarrow 1/3 \rightarrow 0/3.$$

This behaviour is typical for stiff nonlinear PDEs: Newton basins in high dimensions frequently exhibit fractured or fractal boundaries, especially when the Jacobian becomes nearly singular. Because the discretised PDE lives in a 160-dimensional space, a one-dimensional ε -scan naturally intersects this complicated hypersurface in a jagged, non-monotone way.

In contrast, RIFT maintains a stable success region across the same interval, consistent with its Lie-algebraic curvature-injection mechanism.

C.2 Python Code for the ε -Sweep Basin Experiment

```
=====
# GLOBAL PARAMETERS
# =====
N = 80
max_iter = 30
conv_tol = 1e-3
eps_rift = 0.3
newton_step = 1.0
rift_step = 1.0
rng = np.random.default_rng(0)
smallvalueshiftsolution = 0.0 # overwritten during epsilon sweep

x = np.linspace(0.0, 1.0, N + 2)
h = x[1] - x[0]

# -----
# RIFT rotation
# -----
def so2_rotation(vec, angle):
    v = vec.copy()
    if len(v) >= 2:
        c = np.cos(angle)
        s = np.sin(angle)
        v0 = c * v[0] - s * v[1]
        v1 = s * v[0] + c * v[1]
        v[0], v[1] = v0, v1
    return v

# -----
# FD Jacobian
# -----
def jacobian_fd(F, u, param, eps=1e-6):
    n = len(u)
    J = np.zeros((n, n))
    F0 = F(u, param)
    for j in range(n):
        u2 = u.copy()
        u2[j] += eps
```

```

        J[:, j] = (F(u2, param) - F0) / eps
    return J

# -----
# Coupled PDE (u,v)
# -----
def pde2_coupled_uv(state, params):
    lam, alpha, k = params
    u = state[:N]
    v = state[N:]

    u_full = np.zeros(N + 2)
    v_full = np.zeros(N + 2)
    u_full[1:-1] = u
    v_full[1:-1] = v

    res = np.zeros_like(state)

    # u-equation
    for i in range(1, N + 1):
        up = (u_full[i+1] - u_full[i-1]) / (2*h)
        upp = (u_full[i+1] - 2*u_full[i] + u_full[i-1]) / (h*h)
        res[i-1] = -upp + lam * up**3

    # v-equation
    for i in range(1, N + 1):
        up = (u_full[i+1] - u_full[i-1]) / (2*h)
        vpp = (v_full[i+1] - 2*v_full[i] + v_full[i-1]) / (h*h)
        res[N + i - 1] = -vpp + alpha * (up**k) * v_full[i]

    return res

# -----
# Newton solver
# -----
def newton_solver(F, param, u0):
    u = u0.copy() + smallvalueshiftsolution
    for it in range(1, max_iter + 1):
        r = F(u, param)
        if np.linalg.norm(r) < conv_tol:

```

```

        return True
    J = jacobian_fd(F, u, param)
    try:
        delta = np.linalg.solve(J, -r)
    except np.linalg.LinAlgError:
        return False
    u += newton_step * delta
    return False

# -----
# RIFT solver
# -----
def rift_solver(F, param, u0):
    def F_rift(u, p):
        u_lift = so2_rotation(u, eps_rift)
        r_native = F(u_lift, p)
        return so2_rotation(r_native, -eps_rift)

    u = u0.copy() + smallvalueshiftsolution
    for it in range(1, max_iter + 1):
        r = F_rift(u, param)
        if np.linalg.norm(r) < conv_tol:
            return True
        J = jacobian_fd(F_rift, u, param)
        try:
            delta = np.linalg.solve(J, -r)
        except np.linalg.LinAlgError:
            return False
        u += rift_step * delta
    return False

# =====
# CRITICAL EPSILON ZOOM-IN FOR Coupled Hamiltonian-Jacobi-Schrodinger PDE system
# =====
print("\n CRITICAL EPSILON ZOOM-IN Coupled Hamiltonian-Jacobi-Schrodinger PDE")

F = pde2_coupled_uv
param = np.array([399.65957202, 10.49090946, 4])

eps_low = 0.00000045

```



```

eps_high = 0.00000060

num_trials = 3
num_refinements = 12

def success_rate(eps):
    global smallvalueshiftsolution
    smallvalueshiftsolution = eps
    newton_succ = 0
    rift_succ = 0
    for _ in range(num_trials):
        u0 = 0.1 * rng.normal(size=2*N)
        if newton_solver(F, param, u0): newton_succ += 1
        if rift_solver(F, param, u0): rift_succ += 1
    return newton_succ, rift_succ

for step in range(num_refinements):
    mid = 0.5 * (eps_low + eps_high)
    newton_mid, rift_mid = success_rate(mid)

    print(f"step {step+1:02d}: eps={mid:.12f}  Newton={newton_mid}/{num_trials}  RI
FT={rift_mid}/{num_trials}")

    if newton_mid == 0:
        eps_low = mid
    else:
        eps_high = mid

critical_eps = 0.5 * (eps_low + eps_high)
print(f"\nEstimated critical epsilon where New
ton loses basin: {critical_eps:.12f}")

```

C.3 Numerical Results: 12-Step Refinement Table

The output of above python program is shown in table below. Estimated critical epsilon:

$$\varepsilon_{\text{crit}} \approx 4.65326 \times 10^{-7}.$$

Step	ε (midpoint)	Newton Success	RIFT Success
1	5.25000×10^{-7}	2/3	2/3
2	4.87500×10^{-7}	1/3	2/3
3	4.68750×10^{-7}	1/3	1/3
4	4.59375×10^{-7}	0/3	0/3
5	4.64062×10^{-7}	0/3	0/3
6	4.66406×10^{-7}	1/3	1/3
7	4.65234×10^{-7}	0/3	0/3
8	4.65820×10^{-7}	1/3	1/3
9	4.65527×10^{-7}	1/3	1/3
10	4.65381×10^{-7}	1/3	1/3
11	4.65308×10^{-7}	0/3	0/3
12	4.65344×10^{-7}	1/3	0/3

C.4 Concluding Remark

The jagged, fractal-like Newton basin observed here is entirely consistent with classical behaviour of stiff nonlinear PDE solvers and does not challenge the smooth local regularity radius established in Appendix A. the contrast between Newton's fractured global basin and RIFT's stable convergence region provides expresses empirically the geometric mechanism underlying RIFT.

RIFT can be applied to the 3D Incompressible Euler equation. The stationary incompressible Euler equations are

$$(\mathbf{u} \cdot \nabla)\mathbf{u} + \nabla p = 0, \quad \nabla \cdot \mathbf{u} = 0. \quad (\text{C250})$$

These equations require the velocity field to be a volume-preserving diffeomorphism. RIFT can be applied to the discretised stationary Euler equation, particularly near singular configurations where standard Newton methods fail. In such situations, RIFT may have significant numerical advantages, allowing the tracking of solution branches that ordinary Newton iteration cannot follow.